

G2-3 Foundry Feedback Automation — Specification Scratchpad

Purpose: Working document for developing the Gen 2 specification for G2-3 (Foundry Feedback Automation). Captures research, analysis, and the enterprise IT parallel that informs what this capability requires. This is specification development, not intel tracking.

Status: Draft — input to Gen 2 whitepaper

1. The Core Insight: This Architecture Already Exists

The automated field-to-foundry feedback loop is not novel architecture. It is the standard operating model for enterprise IT self-healing, observability-driven remediation, and AIOps. The difference between an attacker and an IT admin is intent, not technique. Both run the same commands on the same systems through the same architectural patterns. Attackers live off the land because the land IS IT operations.

2. The Enterprise IT Self-Healing Loop (Researched)

2.1 The architecture (Dynatrace + ServiceNow + Ansible)

Documented in production at Lockheed Martin, BT Digital, Commerzbank, CareSource. This is the commercially deployed version of what the Hydra's foundry needs to do.

The loop:

Step 1: OBSERVE

Enterprise: Dynatrace agent detects anomaly (service down, memory leak, cert expiring)

Hydra: Field implant discovers environment (EDR type, patch level, topology)

Step 2: DIAGNOSE

Enterprise: Agent runs diagnostic scripts (check logs, resources, connectivity)

Hydra: Implant runs recon (credential harvest, vulnerability scan, service enumeration)

Step 3: REPORT

Enterprise: Sends structured finding to ServiceNow (ITSM ticket with enriched context)

```

Hydra:      Sends structured intelligence report through C2 to foundry

Step 4: ANALYSE
Enterprise: Davis AI performs root cause analysis, matches against knowledge
base
Hydra:      Foundry analyses environment, matches against exploit library

Step 5: DECIDE
Enterprise: Knowledge base + AI selects treatment (more diagnostics OR apply
fix)
Hydra:      Foundry decides: gather more intel OR build custom exploit

Step 6: GENERATE
Enterprise: Ansible Lightspeed generates remediation playbook that didn't exist
before
Hydra:      Foundry AI generates custom exploit payload for this specific
environment

Step 7: EXECUTE
Enterprise: Ansible executes playbook on target system (OS commands, service
restarts, config changes)
Hydra:      C2 deploys custom implant to field agent (lateral move, persistence,
exfil)

Step 8: VERIFY
Enterprise: Agent confirms fix worked, closes ticket, updates CMDB
Hydra:      Implant confirms persistence established, reports success, updates
foundry

```

Every step maps one-to-one. The architecture is identical. The commands at Step 7 are often literally the same — `systemctl restart`, `apt-get update`, `kubectrl apply`, `registry writes`, `scheduled task creation`, `service account modification`.

2.2 Key commercial implementations

Dynatrace + ServiceNow + Ansible (Lockheed Martin) - Dynatrace detects anomaly → creates ServiceNow ticket → Ansible runs remediation playbook → auto-healing - Davis AI provides deterministic root cause analysis - "Auto-baselining, auto-remediation, monitoring as code" - In production at a defence contractor — the irony is not lost

Red Hat Ansible Lightspeed + Event-Driven Ansible - Three generations of automation documented by Red Hat: 1. Task-based: run known playbooks on schedule/trigger 2. Event-driven: react to known conditions with pre-written rules 3. Agent-driven: handle novel situations with real-time contextual judgment - Lightspeed: "AI generates a remediation playbook that didn't exist before" - This IS the foundry generating custom payloads from field intelligence

ServiceNow Now Assist + ITOM - AI agents validate hypotheses, assess business impact, execute full remediation workflows automatically - "Agents transition from reactive troubleshooting to autonomous prevention" - 71% of enterprises deploy AI/AIOps; only 36% consistently automate remediation (theCUBE Research 2025)

Event-Driven Ansible (Red Hat) - "Process events containing discrete intelligence about conditions in the IT environment, determine the appropriate response, then execute automated actions to address or remediate" - Sources include: monitoring tools,

observability platforms, security scanners - Outputs include: running playbooks, creating tickets, sending notifications, self-healing

2.3 The three generations mapping

Generation	Enterprise IT	Hydra Equivalent
1. Task-based	Run known playbooks on schedule	Commodity malware with fixed payload
2. Event-driven	React to known conditions with rules	Shai-Hulud: self-propagating but scripted
3. Agent-driven	Handle novel situations with contextual judgment	The Hydra: autonomous, reasoning, adaptive

Enterprise IT is already at Generation 3 in production. The Hydra's G2-3 is Generation 3 pointed at offensive objectives.

3. What CleverHans Demonstrates vs Doesn't

3.1 What CleverHans gives the foundry loop

CVE integration within hours (intelligence intake): - "Capable of incorporating newly published vulnerabilities within hours of disclosure" - The worm pulls external intelligence and adapts its attack strategy - This is Step 1-2 of the loop: observe and diagnose, with real-time intelligence updates - Maps to: the field agent receiving updated arsenal data

Recursive reasoning per target (adaptive analysis): - "Goal-directed reasoning that adapts to the vulnerabilities of each target" - Each target gets a unique, reasoned attack strategy - This is Step 4-5: the analysis and decision layer working autonomously - Maps to: the foundry analysing the environment and selecting approach

Self-sustaining compute (infrastructure): - "Each compromised machine becomes part of the worm's own infrastructure" - Parasitic resource usage — no external infrastructure cost - Maps to: the foundry running on stolen resources, not requiring separate infrastructure

3.2 What CleverHans does NOT give the foundry loop

No structured field-to-foundry reporting: - The worm does NOT say "I'm stuck, here's what I need, build me something" - It works from a pre-existing CVE library, not custom-built exploits - Missing: the autonomous decision "time to retreat and request help"

No custom exploit generation: - Uses known CVEs and misconfigurations, not novel exploits - No interaction with a foundry for custom builds - Missing: the foundry building something that didn't exist in the arsenal

No OTA deployment of new implants: - The worm doesn't receive and deploy new payloads mid-campaign - No deployment mechanism for foundry-built custom implants - Missing: Step 6-7 of the loop (generate + execute custom payload)

No retreat-and-wait cycle: - Continuously active for 5 days, no dormancy - Doesn't sacrifice early access to wait for custom weapons - Missing: the strategic retreat that the portfolio strategy describes

Summary: CleverHans demonstrates the INTAKE side (Steps 1-5) but not the GENERATION and DEPLOYMENT side (Steps 6-8).

4. Living Off the Land — Threat Hunt Avoidance, C2 Selection, and Board Control

4.1 Living off the land is not just evasion — it's the operating model

Living off the land is typically framed as an evasion technique: use legitimate tools to avoid detection. But in the Gen 2 context, it's much more than that. It's how the AI orchestrator:

- **Profiles the environment** — what tools exist, what scripts run, what RPA workflows execute, what packages are installed, what service accounts operate, what scheduled tasks fire
- **Selects C2 channels** — not "which C2 protocol to use" but "which legitimate activity pattern in THIS environment to mimic." If the target runs Ansible, the C2 looks like Ansible. If they have RPA bots syncing to SharePoint, the exfil looks like an RPA sync
- **Avoids threat hunts** — not by hiding, but by being indistinguishable from the environment. The best hiding place is in plain sight, performing actions that look exactly like what the environment already does
- **Performs mission preparation** — hijacking existing scripts for injection, modifying existing scheduled tasks rather than creating new ones, adding lines to existing RPA workflows, poisoning existing package dependencies
- **Establishes board control** — placing implants at network positions where they become infrastructure, not intruders. A compromised build server isn't malware — it's part of the CI/CD pipeline. A modified monitoring agent isn't an implant — it's an observability tool

The difference between an IT admin and an attacker isn't the command. It's the intent behind the command. And intent is invisible to signature-based detection.

4.2 The chess-piece value model and loss tolerance

Every implant in the Gen 2 portfolio has a value to the campaign. The AI orchestrator must make real-time decisions about what to sacrifice, what to protect, and what constitutes acceptable loss.

Pawns — expendable probes (value: intelligence, not access) - Purpose: test defences, characterise SOC, probe network segmentation, measure response time - Loss tolerance: 100% expected. Every pawn is deployed with the assumption it will be caught - Living off the land: pawns can be INTENTIONALLY noisy. They don't need to hide — they need to trigger responses that the rooks observe - Loss impact: zero. The intelligence

gathered exceeds the cost of the implant - Detection response: SOC thinks "we caught malware, contained it." Campaign continues

Rooks — board control nodes (value: persistent access, network position) -

Purpose: maintain long-term access, provide re-entry points, serve as observation posts, relay C2 for other pieces - Loss tolerance: low. Losing a rook is meaningful but not fatal. The campaign is weakened but not over - Living off the land: rooks MUST live off the land. They survive by being invisible — memory-resident, mimicking legitimate processes, using existing scheduled tasks, blending into normal service account activity - Loss impact: significant. A rook provides network position that takes time to re-establish. Losing a rook on the build server means losing domain admin access. Losing a rook on an unmonitored segment means losing deep network persistence - Detection response: SOC escalates to threat hunt. But other rooks survive because they're in different network positions with different C2 channels

King — mission objective (value: the reason the campaign exists) - Purpose: crown jewel exfiltration, supply chain compromise, destructive action - Loss tolerance: the king is never sacrificed. If the king is compromised, the mission fails - Living off the land: the king's actions are the most carefully disguised. Exfiltration looks like backup sync. Data staging looks like normal ETL. The king only activates when conditions are optimal - Loss impact: checkmate. Campaign failure. The strategic objective is not achieved

The orchestrator's decision tree:

For each implant, the AI orchestrator continuously evaluates:

1. What is this implant's role? (pawn / rook / king)
2. What is its current detection risk? (based on environment telemetry)
3. Is it still providing value? (intelligence / access / position)
4. Should it act, wait, or self-destruct?

PAWN decisions:

IF detection_risk HIGH → accept detection (that's the purpose)
IF intelligence_gathered SUFFICIENT → report and self-destruct
IF SOC_response OBSERVED → record SOC tier and techniques, report to rooks

ROOK decisions:

IF detection_risk INCREASING → reduce activity (longer beacon intervals, less movement)
IF threat_hunt DETECTED → go completely dormant (zero activity until hunt ends)
IF rook_lost IN OTHER_POSITION → assess whether to activate backup rook or request new implant from foundry
IF access_path BURNED → report to foundry: "need new lateral move technique for this environment"

KING decisions:

IF all_rooks_confirmed STABLE → assess exfiltration window
IF SOC_distracted (dealing with pawn detection) → optimal activation window
IF exfiltration_complete → cleanup traces, optionally leave dormant rook for future
IF detection_risk CRITICAL → abort, preserve rooks, retreat to dormancy

4.3 How living off the land connects to every Gen 2 capability

This isn't isolated to evasion. Living off the land is the operating principle that connects all four Gen 2 capability groups through the AI orchestration layer:

G2-1 (Portfolio Orchestration) ← decides WHAT lives off the land - The orchestrator profiles the environment's legitimate tools, scripts, RPA workflows, and service accounts - Assigns each implant a disguise based on what's available: "this rook will impersonate the Ansible service account, this rook will inject into the monitoring agent, this pawn will run as a scheduled PowerShell task" - Coordinates: no two implants use the same disguise (avoids correlated detection) - Decides implant placement for board control: build servers, jump boxes, monitoring infrastructure — positions that provide both access AND natural cover

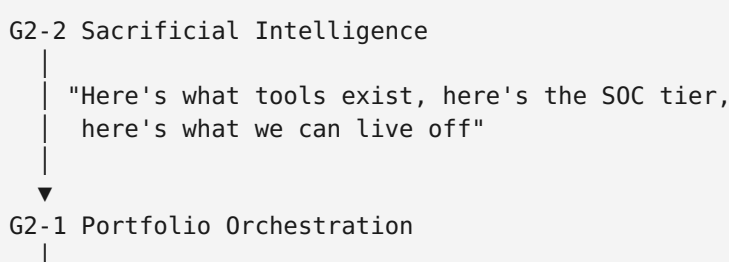
G2-2 (Sacrificial Intelligence) ← uses pawns to learn WHAT to live off - Pawns probe the environment to discover: what EDR is installed, what legitimate tools exist, what the SOC responds to - The intelligence feeds directly into the orchestrator's living-off-the-land strategy: "CrowdStrike is on these hosts but not those. Ansible runs nightly. The RPA bot syncs to SharePoint every 2 hours. The build service uses AWS credentials stored in environment variables" - This IS the environment profiling that makes living off the land targeted rather than generic

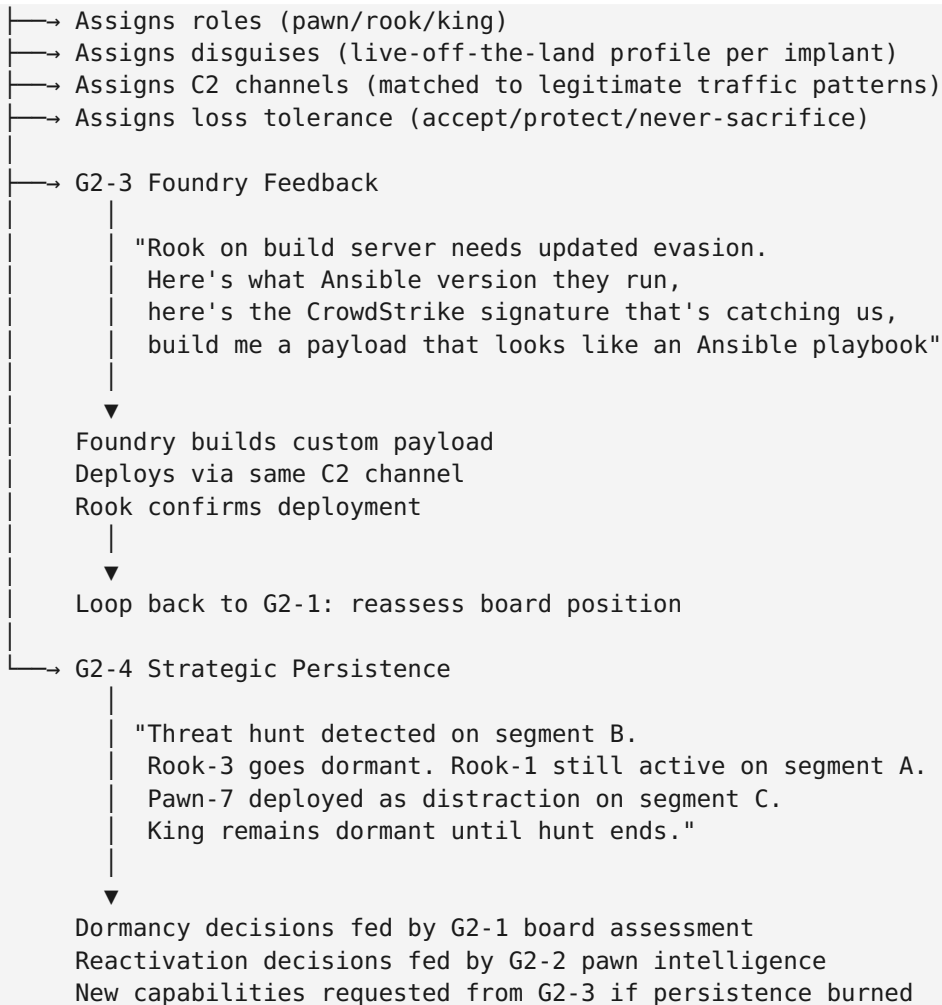
G2-3 (Foundry Feedback) ← builds tools that live off the land - When a rook reports "my current technique is being detected," the foundry response isn't "here's a new malware binary" — it's "here's a modified version of the legitimate script that already runs on this host, with your payload injected" - The foundry generates payloads that are indistinguishable from the target's own tooling because the field intelligence tells the foundry exactly what tooling exists - The enterprise IT parallel: Ansible Lightspeed generates a playbook that didn't exist before, but it looks exactly like an Ansible playbook because it IS one. The Hydra's foundry generates an exploit that looks exactly like a legitimate script because it's built FROM the target's own scripts

G2-4 (Strategic Persistence) ← survives because it lives off the land - Dormancy works because the dormant implant IS a legitimate process / service / scheduled task - Threat hunt evasion works because the implant's activity baseline matches the legitimate baseline - Reactivation works because "service account resumes normal activity" is indistinguishable from "implant resumes C2" - The rook survives the threat hunt not by hiding from it, but by being part of the furniture

4.4 The interconnection model — Gen 2 is a system, not a list

The critical specification requirement: Gen 2 capabilities cannot be assessed independently. They form an integrated decision system where each capability feeds the others:





The AI orchestration layer is the connective tissue. It doesn't sit in one Gen 2 capability — it spans all four, making real-time decisions about:

- What to sacrifice (G2-2 input → G2-1 decision)
- What to preserve (G2-4 dormancy → G2-1 board control)
- What to request (G2-3 foundry → G2-1 capability gap)
- When to act vs wait (all four capabilities informing timing)

4.5 The detection implication restated

If every Gen 2 capability operates through living off the land, and the AI orchestrator coordinates them as an integrated system, then:

- Sacrificial pawns look like commodity malware (SOC classifies as "standard incident")
- Persistent rooks look like legitimate services, scripts, RPA bots, monitoring agents
- Foundry payloads look like legitimate Ansible playbooks, PowerShell scripts, scheduled tasks
- C2 traffic looks like normal SaaS API calls, cloud sync, backup traffic
- The king's exfiltration looks like normal data movement (ETL, backup, reporting)

The only detection method that survives this is behavioural analytics with environmental context — understanding not just what's happening but whether what's happening makes

sense for THIS user, THIS service account, THIS time of day, THIS data volume, in THIS business context.

And that's the capability most organisations are missing.

5. G2-3 Specification Requirements (Draft)

5.1 What G2-3 must demonstrate to be classified as a Gen 2 capability

G2-3.1: Autonomous Field Intelligence Reporting

The field agent must: - Autonomously decide "I have gathered enough intel" or "I've hit a wall I can't solve" - Structure a machine-readable intelligence report containing: environment profile (OS, patch level, EDR, topology), access profile (current privilege, available credentials, lateral move paths), defensive posture (SOC tier, detection methods, response latency), and capability gap (what it needs that it doesn't have) - Send the report through existing C2 without human instruction - Optionally: retreat to dormancy after reporting (the strategic retreat from the portfolio strategy)

G2-3.2: Foundry Analysis and Custom Generation

The foundry must: - Receive and parse the structured field intelligence autonomously - Match the environment against known exploit library (CVE matching for patch gaps) - Where no known exploit exists: generate custom payload (evasion tuned for the specific EDR, C2 tuned for the specific traffic patterns, exploit tuned for the specific vulnerability) - Validate the custom payload (sandbox test against replicated environment based on field intel) - Package for deployment

G2-3.3: OTA Deployment and Closed-Loop Verification

The deployment must: - Push the custom payload through existing C2 channels (same channels as operational C2 — no separate infrastructure) - Field agent receives, validates, and deploys without human instruction - Optionally: staged deployment (test in low-risk area before deploying against crown jewels) - Field agent confirms success or failure - Failure: loop back to G2-3.1 (report updated intelligence, request revised payload) - Success: advance to next phase (lateral move, persistence, collection)

5.2 The minimum viable G2-3

The full autonomous loop is the specification. But what's the minimum that would count as Gen 2 system integration?

Option A: Structured request, human foundry - Field agent autonomously structures and sends intelligence report - Human analysts receive, analyse, build custom exploits (2-4 weeks) - Custom payload deployed via C2 - This is arguably how Tier 1 actors already operate (manually) - Verdict: this is Gen 1 with better tooling, not Gen 2

Option B: Structured request, AI-assisted foundry, human approval - Field agent autonomously structures report - AI analyses and generates candidate exploit/evasion -

Human reviews and approves deployment - Custom payload deployed via C2 - Verdict: hybrid — approaching Gen 2 but human-in-the-loop means not fully integrated

Option C: Fully autonomous loop - Field agent structures report autonomously - Foundry AI analyses, generates, validates, and deploys without human intervention - Field agent receives and executes - Closed-loop verification - Verdict: this is Gen 2. Full system integration. The weapon system operates end-to-end

Recommendation: Gen 2 requires Option C. Options A and B are Gen 1 maturity levels (manual or AI-assisted component use). Gen 2 is system integration — the loop runs without human intervention. The human operator sets the strategic objective (Gen 1 operator control plane), but the tactical loop executes autonomously.

5.3 What each step requires from Gen 1

G2-3 STEP	DEPENDS ON GEN 1 COMPONENT	GEN 1 STATUS
Field recon	ia-1 (vuln scanning)	EXISTS
	ia-2 (exploit execution)	EXISTS
	ao-2 (tool selection)	PARTIAL
Intelligence reporting	sd-3 (field-to-foundry)	GAP ← the bottleneck
	ao-5 (field intel integration)	PARTIAL (CleverHans)
Foundry analysis	ao-1 (attack surface assessment)	EXISTS
	ao-3 (value assessment)	GAP
Custom generation	ia-5 (zero-day weaponisation)	PARTIAL (GTIG)
	ev-1 (polymorphic code)	EXISTS
	ev-3 (AI evasion adaptation)	PARTIAL (PROMPTSPY)
OTA deployment	sd-4 (OTA via C2)	PARTIAL
	c2-1 (legitimate cloud C2)	EXISTS
	c2-4 (dynamic C2 rotation)	PARTIAL (PROMPTSPY)
Verification	ao-5 (field intel integration)	PARTIAL
	sd-3 (feedback reporting)	GAP ← same bottleneck

The bottleneck is sd-3 in Gen 1 — the autonomous request structuring. Everything else is PARTIAL or EXISTS. The loop is blocked by one component.

6. The Enterprise IT Parallel — Evidence Catalogue (Context, Not Tracking Evidence)

These are recorded as context for the Gen 2 whitepaper. They explain WHY G2-3 is feasible and WHY the architecture isn't novel. They do NOT change Gen 1 gap status.

Tool/Platform	What it demonstrates	G2-3 equivalent
Dynatrace Davis AI	Deterministic root cause analysis from agent telemetry	Foundry analysis of field intelligence

Tool/Platform	What it demonstrates	G2-3 equivalent
ServiceNow ITOM + Now Assist	AI agents executing autonomous remediation workflows	Foundry generating and deploying treatments
Ansible Lightspeed	AI generates remediation playbooks that didn't exist before	Foundry generating custom exploits from field intelligence
Event-Driven Ansible	Event → rule matching → automated action execution	Field trigger → foundry response → deployment
Red Hat Insights	Detects issue → emits event → triggers remediation → closes loop	Full closed-loop field-to-foundry-to-field
Dynatrace + ServiceNow integration	Enriched problems pushed to ITOM, AI agents trigger remediation	Structured intel → foundry analysis → automated response
Lockheed Martin self-healing stack	Dynatrace + ServiceNow + Ansible: auto-baseline, auto-remediate, monitoring as code	Full G2-3 loop in production at a defence contractor

Key quotes from enterprise documentation:

- Red Hat: "Instead of hand-coding a rule for every possible failure mode, a single intelligent workflow captures the event, uses AI to diagnose the root cause, and generates the remediation dynamically"
- Dynatrace/ServiceNow: "AI agents validate hypotheses, assess business impact, and execute full remediation workflows automatically"
- Red Hat: "The industry is moving through three generations: task-based, event-driven, and agent-driven automation"
- ServiceNow: "AI-powered IT Service & Operations Management enabling IT teams to move beyond traditional operations into proactive systems that continuously learn, adapt, and self-heal"
- Event-Driven Ansible: "Process events containing discrete intelligence about conditions in the IT environment, determine the appropriate response, then execute automated actions to address or remediate"

7. Open Questions for the Gen 2 Whitepaper

1. **Foundry location:** Does the foundry run on stolen infrastructure (like CleverHans's parasitic compute) or on attacker-controlled infrastructure? Stolen infrastructure = no separate C2 footprint but limited compute. Attacker infrastructure = more capability but detectable C2.
2. **Foundry latency:** The enterprise IT loop runs in minutes (Event-Driven Ansible trigger → playbook execution). The Hydra's foundry loop for custom exploit generation might take hours. Is there a minimum latency threshold for Gen 2? Or is the requirement just "faster than the manual process" (currently 2–4 weeks)?
3. **Foundry model:** Single-model (one LLM does analysis, generation, and validation) or multi-agent (specialist agents for each phase, like PentAGI's orchestrator/

researcher/developer/executor)? The enterprise IT parallel uses specialist tools (Dynatrace for observation, ServiceNow for workflow, Ansible for execution). The Hydra might benefit from the same specialisation.

4. **Failure handling:** What happens when the foundry generates an exploit that doesn't work? Enterprise IT has rollback. The Hydra's equivalent is: field agent reports failure → foundry iterates → new payload deployed. How many iteration cycles before the system decides "this environment is too hard, move to next target"?
5. **Quality vs speed trade-off:** Enterprise IT prioritises reliability (Ansible Lightspeed validates before deployment). The Hydra prioritises speed (custom exploit within hours). These are in tension. Does the Gen 2 specification require validation, or is "deploy and test in the field" acceptable?

8. The G2-3 Specification Thesis (For the Whitepaper)

The automated field-to-foundry feedback loop is not a novel capability requirement. It is the standard architecture for enterprise IT self-healing — observe, diagnose, report, analyse, generate, deploy, verify — with three constraints removed: approval gates, intent validation, and audit guardrails.

Every component of this loop exists in commercial production. Dynatrace observes. ServiceNow analyses and orchestrates. Ansible Lightspeed generates novel remediations. Event-Driven Ansible deploys autonomously. The closed loop runs at Lockheed Martin, BT Digital, Commerzbank, and CareSource today.

The offensive version requires the same architecture pointed at exploitation instead of remediation, running through adversarial C2 instead of ServiceNow APIs, and generating exploits instead of playbooks. The commands at the execution layer are often identical — the same OS commands, the same service management, the same infrastructure manipulation that IT admins run every day.

This is why attackers live off the land. The land IS IT operations. The foundry loop IS the self-healing loop. The weapon IS the tool.

Gen 2 G2-3 is classified as achieved when the full loop — from autonomous field intelligence reporting through custom payload generation to closed-loop deployment and verification — operates without human intervention in an offensive context. The architecture exists commercially. The offensive weaponisation is the remaining threshold.

Scratchpad document — input to Gen 2 whitepaper. Not for publication in current form.